Locksmith Lab

Design

Nikola Panayotov

Table of Contents

1. Introduction 3
1.1 Purpose 3
1.2 Design objectives 3
2. Technology choices 3
3. System architecture 3
3.1 Overall arrangement 3
3.2 Backend layers and dependencies 4
4. Frontend design 6
5. Database design 6
5.1 Unsecured DB_Diagram 6
5.2 Secured DB_Diagram 6
6. REST API design 7
7. Unsecured design 7
7.1 Intended usage 7
7.2 Existing weaknesses visible in the code 7
8. Secured design 7
8.1 Security controls 7
8.2 Authorization model 8
8.3 Security implementation status 8
9. Deployment design 8
9.1 Unsecured environment 8
9.2 Secured environment 9
10. Design verification 10
11. Conclusion 10
References 11

1. Introduction

1.1 Purpose

This document specifies the technical design of SecureNotes, a small networked web application for demonstrating how common application and server vulnerabilities arise and how their risks can be reduced.

The project delivers an Unsecured version for controlled vulnerability demonstrations, and a Secured version that addresses the identified weaknesses. The Unsecured application must remain isolated; the Secured application is intended for deployment on an Ubuntu VPS behind Nginx.

1.2 Design objectives

SecureNotes must support registration, login, logout, and creating, viewing, editing and deleting personal notes. The design must separate frontend, API and database responsibilities, make security tests reproducible, and support a before-and-after comparison using the same test cases.

2. Technology choices

Component	Technology	Role
Frontend	React + TypeScript	Browser UI
API	Node.js + Express + TypeScript	Routes, sessions, application logic
Database	PostgreSQL	Users and notes
Containers	Docker	Reproducible environment
Reverse proxy (Secured)	Nginx	HTTPS and request forwarding
Host (Secured)	Ubuntu VPS	Deployment environment

3. System architecture

3.1 Overall arrangement

The frontend (React, n.d.) sends HTTP requests to the Express REST API (Express, n.d.). The API enforces application rules and uses repositories to query PostgreSQL (PostgreSQL Global Development Group, n.d.). For the Secured deployment, Nginx (Nginx, n.d.) terminates HTTPS and forwards API traffic internally.

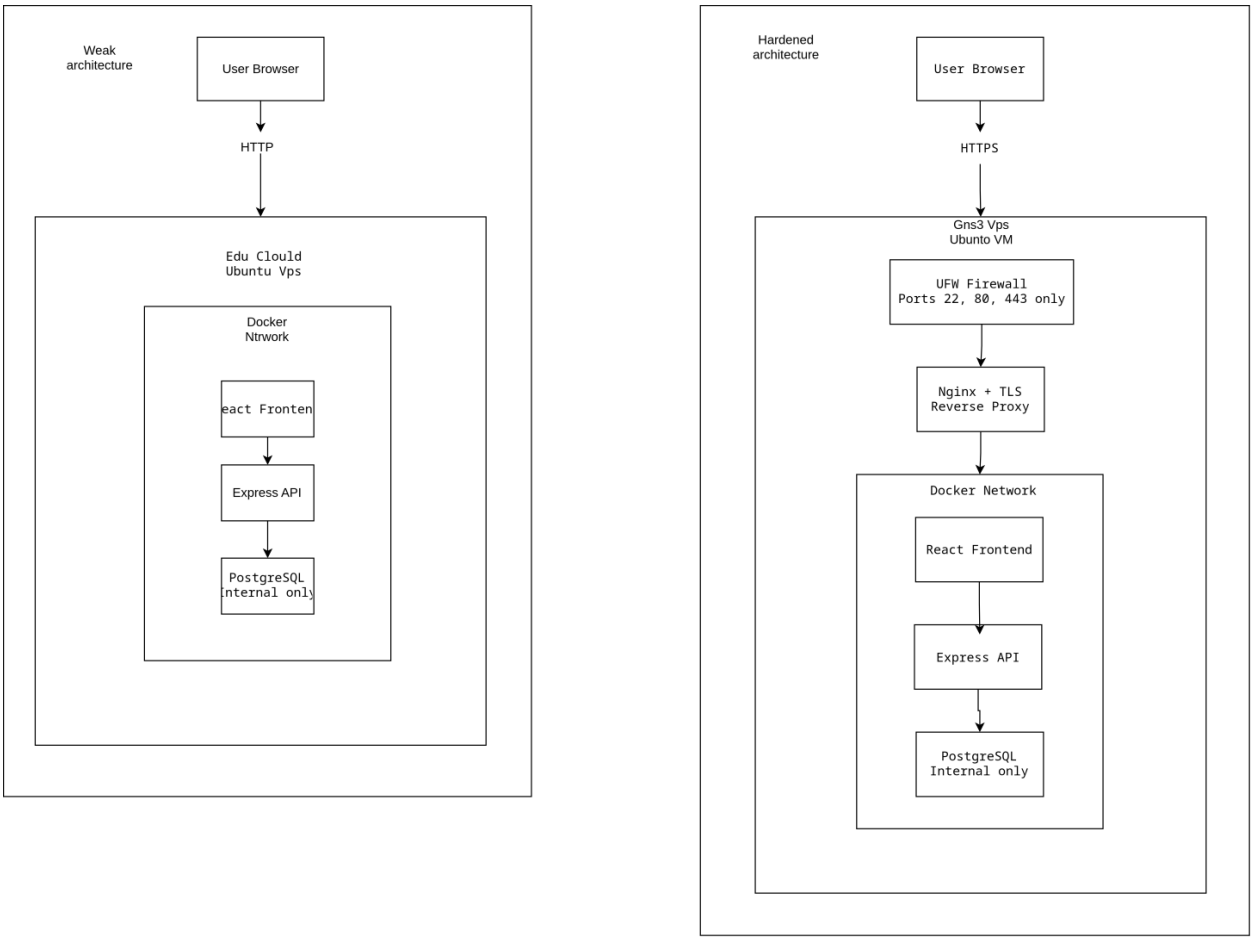


Figure 1. System architecture diagram.

3.2 Backend layers and dependencies

Layer	Purpose
Routes	Map endpoints to controllers; attach middleware
Authentication middleware	Check that the request has an authenticated session
Controllers	Read requests and create HTTP responses
Services	Implement authentication and note operations
Repositories	Execute SQL against PostgreSQL

Authentication flow: AuthRoutes → AuthController → AuthService → UserRepository → PostgreSQL. Note flow: NoteRoutes → NoteController → NoteService → NoteRepository → PostgreSQL. NoteRoutes uses AuthMiddleware, which reads Express Session; it does not directly query the user or note tables.

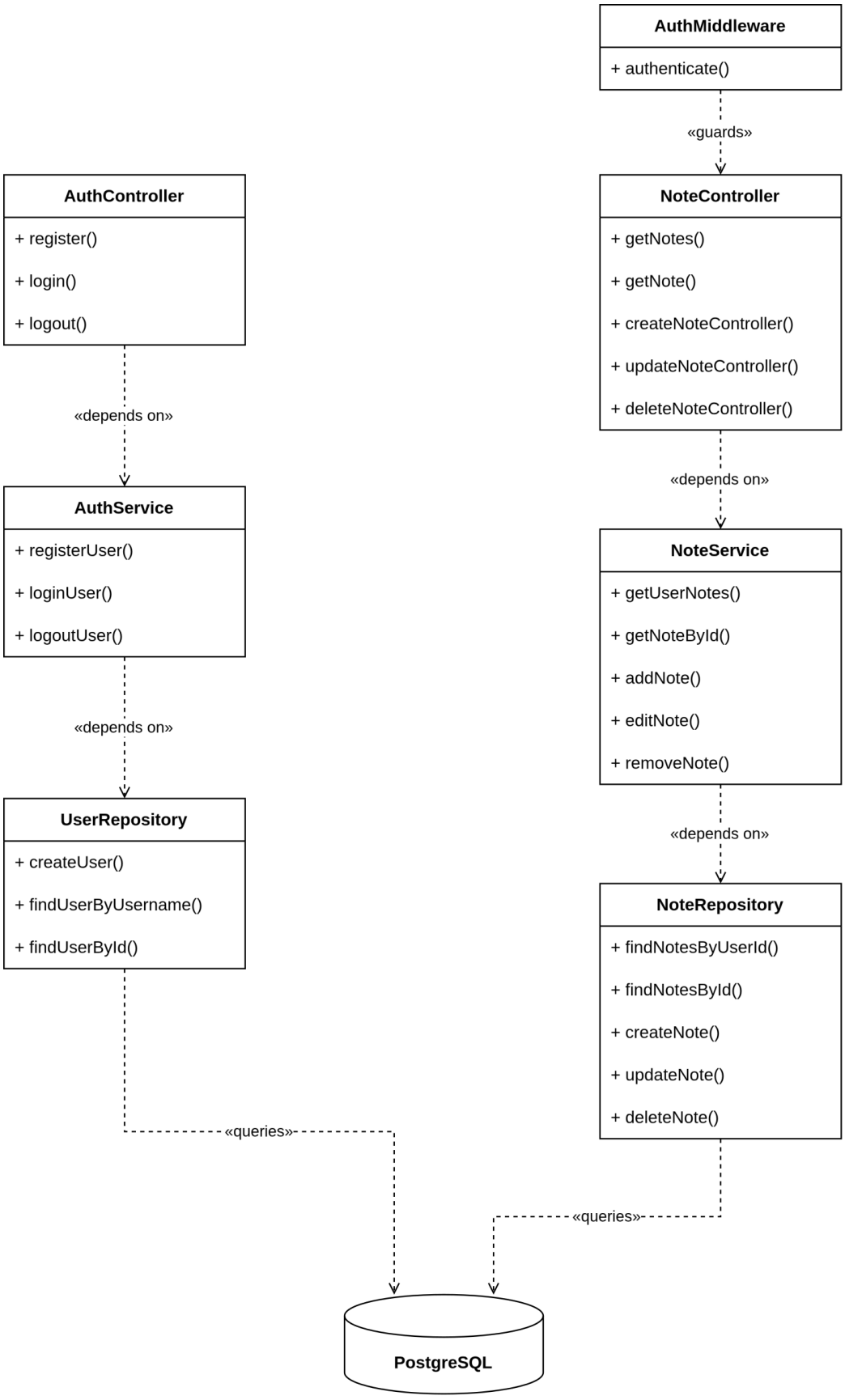


Figure 2. UML diagram of application components and dependencies.

4. Frontend design

Component	User interaction
RegisterPage	Create an account
LoginPage	Enter credentials and start a session
NotesPage	Display and manage personal notes
App	Switch between login, registration and notes views

The frontend handles presentation and API calls only. Authorization decisions must be enforced by the backend; hiding or disabling a button in the browser is not an access-control mechanism.

5. Database design

A user can own multiple notes. A note belongs to one user. The notes.user_id foreign key uses ON DELETE CASCADE.

5.1 Unsecured DB_Diagram

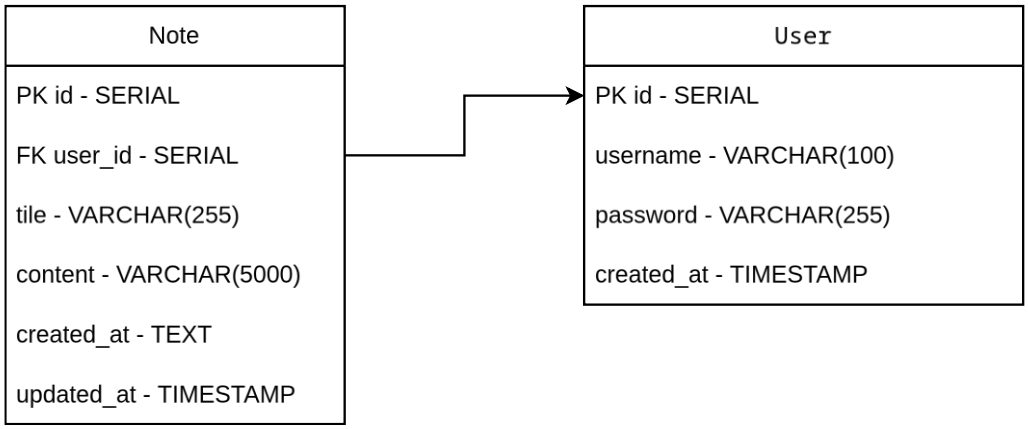


Figure 3. Unsecured database diagram.

5.2 Secured DB_Diagram

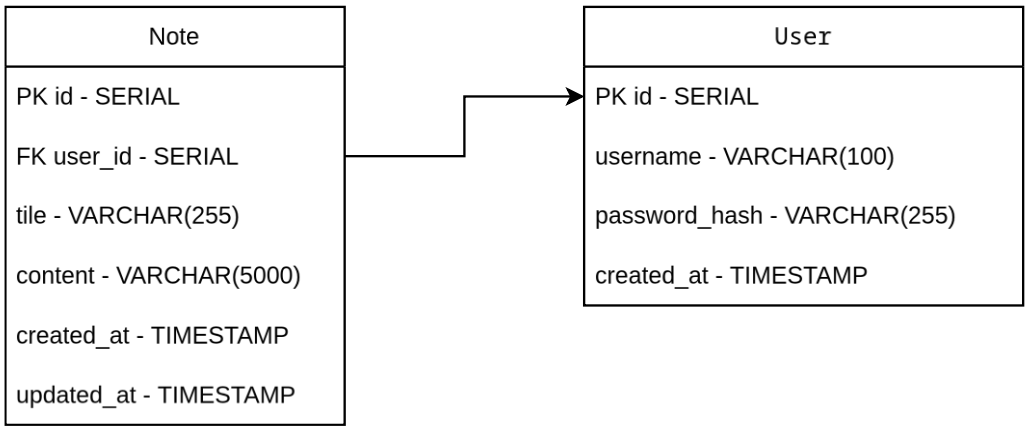


Figure 4. Secured database diagram.

6. REST API design

Method	Endpoint	Purpose
POST	/api/auth/register	Register
POST	/api/auth/login	Login
POST	/api/auth/logout	Logout
GET	/api/notes	List notes
GET	/api/notes/:id	Retrieve note
POST	/api/notes	Create note
PUT	/api/notes/:id	Update note
DELETE	/api/notes/:id	Delete note
GET	/api/health	Database health check

All note routes use the authenticate middleware. In the existing Unsecured implementation, middleware checks for req.session.userId, while certain note operations do not check whether the requesting user owns the note. The Secured version must enforce ownership in note-specific database operations.

7. Unsecured design

7.1 Intended usage

The Unsecured version is intentionally weak and operates only on locally controlled or otherwise authorized isolated infrastructure. Its purpose is to reproduce weaknesses and document their effect, not to serve real users on the public internet.

7.2 Existing weaknesses visible in the code

Area	Existing design	Risk
SQL queries	User and note repositories interpolate values into query strings	SQL injection
Authentication	Credentials are compared in SQL	Authentication bypass if query is manipulated
Passwords	Passwords stored without hashing	Credential disclosure
Authorization	Note lookup, update and deletion do not consistently scope by session user	Cross-user access
Session cookie	Current configuration has httpOnly: false and secure: false	Session exposure
Transport	Local HTTP is used during development	Unencrypted traffic

Cross-site scripting and other input validation issues are planned test areas (OWASP Foundation, 2021), but their exploitability must be established through testing rather than assumed from this design.

8. Secured design

8.1 Security controls

Area	Planned control
SQL injection	Parameterized queries with bound values

Authentication	Secure password verification, session renewal and strong session secrets
Passwords	Appropriate one-way password hashing
Authorization	Scope all note-specific operations to the authenticated session user
Validation	Server-side validation of field types, lengths and identifiers
XSS	Safe text rendering and avoidance of unsafe HTML injection
Cookies	HttpOnly, Secure and appropriate SameSite settings
Network	HTTPS via Nginx; PostgreSQL not publicly exposed
Host	UFW with only required ports accessible
Monitoring	Relevant error and security-event logging without secrets

8.2 Authorization model

The API obtains the user ID from the server-side session (OWASP Foundation, n.d.). When viewing, updating or deleting a note, both the note ID and authenticated user ID are checked. Client-supplied owner IDs are never accepted as proof of ownership. Unauthorized requests must not return another user's note contents.

8.3 Security implementation status

These are design requirements for the Secured version. They should be checked against the completed implementation and supported with validation evidence; they are not presented here as already implemented.

9. Deployment design

9.1 Unsecured environment

Run Unsecured only on an isolated Docker network (Docker, n.d.), local VM or authorized EDU Cloud environment. Limit access to test users and test systems. Do not expose its intentionally vulnerable services to the public internet.

Figure 5 shows the Unsecured deployment. The tester's browser and test tools reach the application over plain HTTP from inside the isolated lab host; there is no route or port forwarding from the public internet to the Docker network.

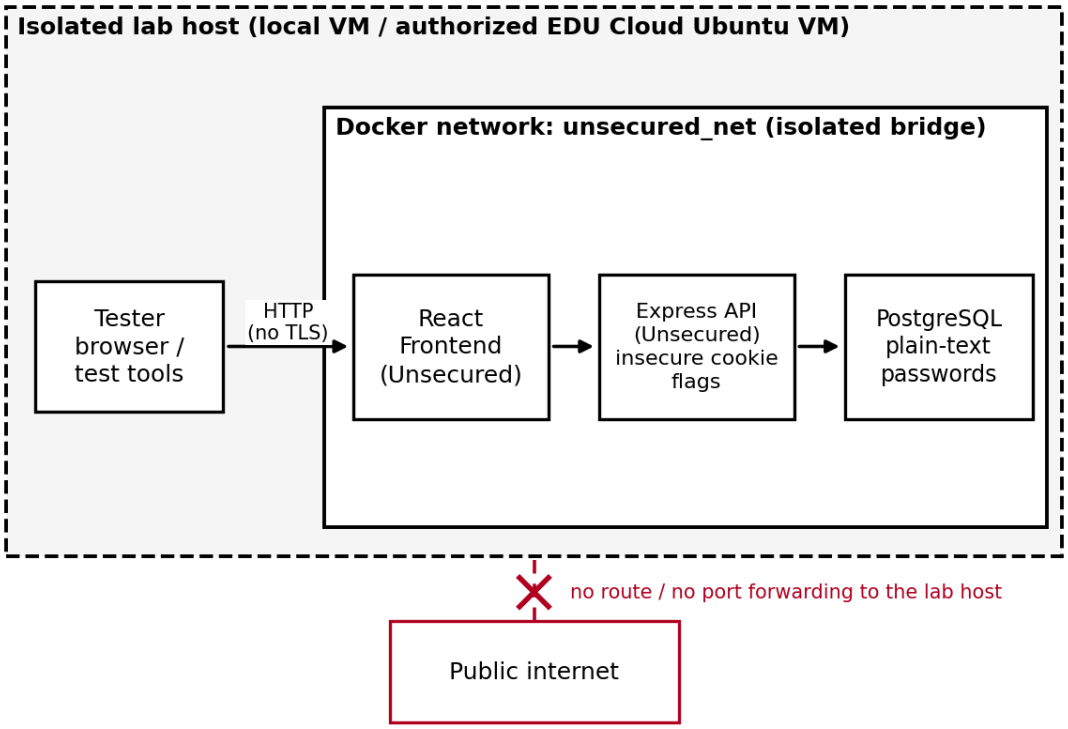


Figure 5. Unsecured deployment environment.

9.2 Secured environment

Deploy Secured on Ubuntu VPS. Nginx acts as reverse proxy and HTTPS entry point; the Express API and PostgreSQL should use restricted internal communication. Keep credentials and session secrets in deployment configuration outside the repository. Apply firewall rules with UFW (Ubuntu, n.d.).

Figure 6 shows the Secured deployment. Only ports 22 (SSH), 80 (HTTP, redirected to HTTPS) and 443 (HTTPS) pass the UFW firewall. Nginx terminates TLS and forwards requests to the frontend and API over the internal Docker network. PostgreSQL has no published port, and secrets are supplied through deployment configuration rather than the repository.

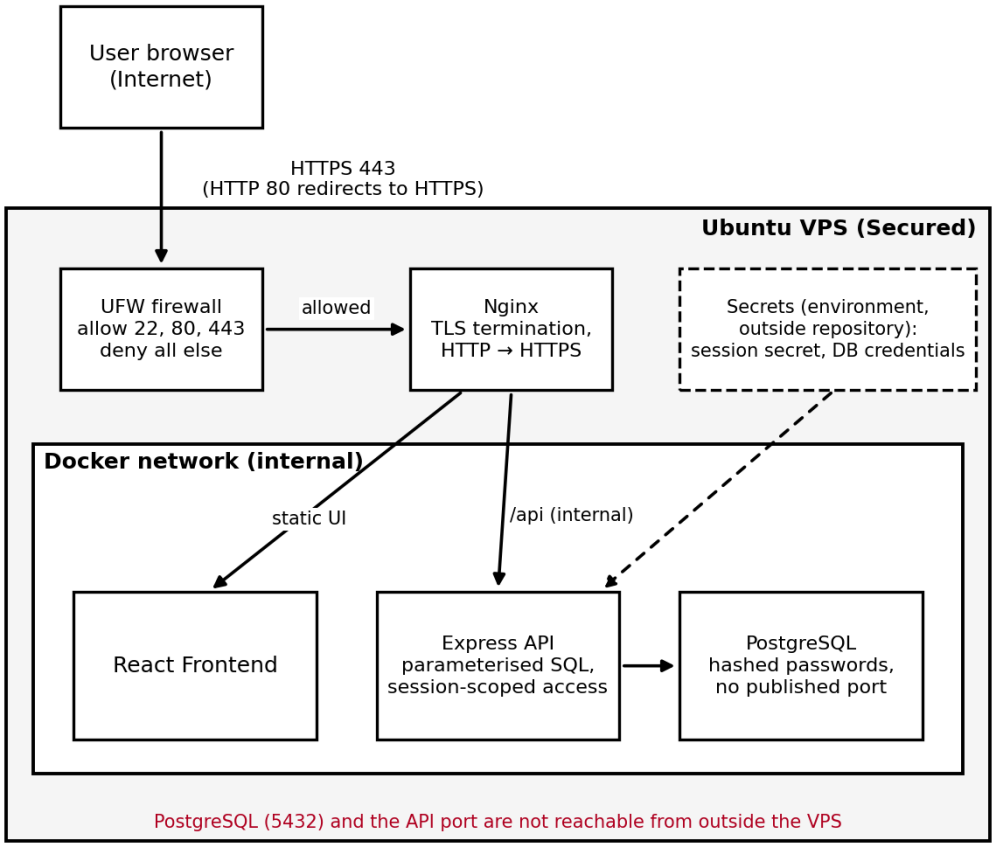


Figure 6. Secured deployment environment.

10. Design verification

ID	Requirement	Evidence
D1	Registration and login function	Functional tests
D2	Note CRUD functions	Functional tests
D3	Unsecured vulnerabilities reproducible in isolation	Test case evidence
D4	Secured SQL injection controls work	Retest injection scenarios
D5	Secured note authorization enforced	Two-user ownership tests
D6	Passwords protected at rest	Inspect stored representation
D7	Secured network exposure limited	Port and configuration checks
D8	Same scenarios compared between versions	Before/after test summary

11. Conclusion

SecureNotes uses a layered frontend, API and PostgreSQL architecture. Unsecured provides a restricted environment for demonstrating vulnerabilities. Secured is designed to preserve the same core functionality while reducing identified risks. Implementation and validation evidence will establish whether the Secured design objectives have been achieved.

References

Docker. (n.d.). *Docker docs*. https://docs.docker.com/

Express. (n.d.). *Express - Node.js web application framework*. https://expressjs.com/

Nginx. (n.d.). *Nginx documentation*. https://nginx.org/en/docs/

OWASP Foundation. (n.d.). *OWASP Cheat Sheet Series*. https://cheatsheetseries.owasp.org/

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

React. (n.d.). *React*. https://react.dev/

Ubuntu. (n.d.). *UFW*. https://help.ubuntu.com/community/UFW